

**LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN
APLIKASI BERBASIS WEB
Modul 4: Mengoperasikan Database dan Generate Data Otomatis
dengan Laravel**



**Universitas
Telkom**

Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S

NIM: 707012530010

Kelas: D4SIKC-49-04

Mata Kuliah: Pengembangan Aplikasi Berbasis Web

**UNIVERSITAS
TELKOM FAKULTAS
ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA
CERDAS TAHUN AJARAN 2026**

Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

Menghubungkan project Laravel dengan database MySQL melalui pengaturan pada file .env.

Membuat tabel database memakai file migration beserta method up() dan down().

Membuat model Eloquent yang terhubung ke tabel mahasiswa.

Membuat controller, route, dan view supaya data dari form bisa disimpan ke database.

Menampilkan kembali data yang sudah tersimpan pada halaman daftar mahasiswa.

Mengisi tabel dengan data dummy secara otomatis memakai seeder dan library Faker.

Alat dan Bahan

Praktikum ini dikerjakan pada laptop dengan spesifikasi berikut.

Perangkat Keras

Laptop dengan prosesor 13th Gen Intel Core i7-13620H.

RAM 16 GB (terbaca 15,7 GB).

Sistem operasi Windows 11 Home Single Language (build 10.0.26200).

Perangkat Lunak

XAMPP 8.2.12, di dalamnya terdapat Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1.

Composer 2.10.3 sebagai dependency manager PHP.

Laravel Framework 12.69.3.

Visual Studio Code 1.140.0 sebagai teks editor.

Google Chrome 154.0.8037.97 untuk menjalankan aplikasi di browser.

Git 2.53.0 untuk version control.

C. Langkah-Langkah Praktikum

Praktikum dikerjakan secara berurutan, mulai dari menyiapkan koneksi database sampai mengisi tabel dengan data dummy. Setiap file disimpan sebelum lanjut ke langkah berikutnya.

C.1 Konsep Migration Laravel

Migration adalah fitur Laravel yang mencatat riwayat perubahan struktur database, kurang lebih

seperti version control system untuk database. Setiap perubahan tabel ditulis dalam satu file migration, sehingga struktur database bisa dibangun ulang di komputer lain hanya dengan menjalankan perintah migrate. File migration memiliki dua method, yaitu up() untuk membuat atau mengubah tabel dan down() untuk membatalkan perubahan saat rollback.

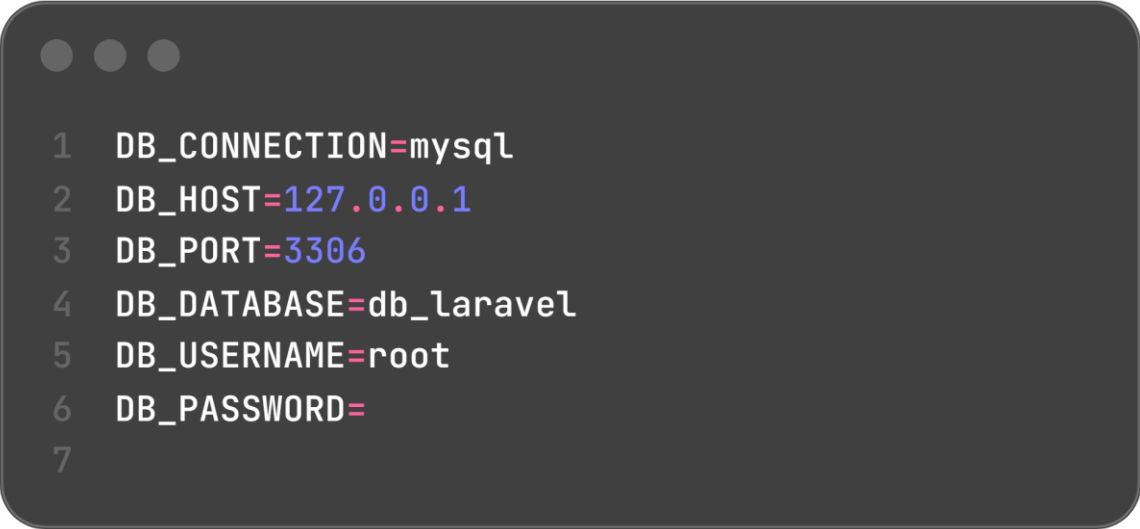
Ada beberapa alasan migration dipakai. Perubahan tabel bisa dibuat dan diubah dengan cepat, file migration bisa dikirim ke programmer lain supaya struktur database tetap konsisten, dan programmer tidak perlu lagi melakukan export atau import file SQL secara manual. Kalau terjadi kesalahan, perubahan terakhir bisa dikembalikan dengan perintah migrate:rollback.

C.2 Konfigurasi Database

Langkah pertama adalah menyalakan Apache dan MySQL pada XAMPP Control Panel sampai keduanya berwarna hijau. Apache berperan sebagai web server, sedangkan MySQL menyimpan data aplikasi.

Setelah itu saya membuka phpMyAdmin di <http://localhost/phpmyadmin>, memilih menu New, lalu membuat database baru bernama db_laravel. Database ini masih kosong karena tabelnya akan dibuat melalui migration.

Koneksi database diatur pada file .env di folder utama project. Bagian yang saya ubah adalah konfigurasi DB seperti berikut.



```
1 DB_CONNECTION=mysql
2 DB_HOST=127.0.0.1
3 DB_PORT=3306
4 DB_DATABASE=db_laravel
5 DB_USERNAME=root
6 DB_PASSWORD=
7
```

Nilai DB_CONNECTION diubah menjadi mysql karena Laravel versi terbaru secara bawaan memakai sqlite. DB_DATABASE diisi db_laravel, DB_USERNAME diisi root, dan DB_PASSWORD dibiarkan kosong mengikuti pengaturan default XAMPP.

C.3 Membuat Migration

Tabel mahasiswa dibuat lewat migration, bukan langsung di phpMyAdmin. Saya menjalankan perintah berikut di terminal.

```
php artisan make:migration create_mahasiswa_table
```

Perintah itu membuat file baru di folder database/migrations dengan nama berawalan tanggal dan waktu, contohnya 2026_10_05_013631_create_mahasiswa_table.php. Awalan waktu tersebut dipakai Laravel untuk menentukan urutan eksekusi migration.

Isi file migration kemudian saya ubah menjadi seperti di bawah ini.

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('mahasiswa', function (Blueprint $table) {
15             $table->id();
16             $table->string('nama');
17             $table->string('email')->unique();
18             $table->string('jurusan');
19             $table->string('umur');
20             $table->timestamps();
21         });
22     }
23
24     /**
25      * Reverse the migrations.
26      */
27     public function down(): void
28     {
29         Schema::dropIfExists('mahasiswa');
30     }
31 };
32
```

Method up() berisi Schema::create('mahasiswa', ...) yang membuat tabel mahasiswa beserta kolomnya. Kolom id() menjadi primary key, nama bertipe string, email bertipe string dan unik, jurusan bertipe string, umur bertipe string, dan timestamps() menambahkan kolom created_at serta updated_at. Method down() berisi Schema::dropIfExists('mahasiswa') untuk menghapus tabel saat rollback.

Setelah file disimpan, migration dijalankan dengan perintah `php artisan migrate`. Perintah ini mengeksekusi semua file migration yang belum pernah dijalankan. Selain tabel mahasiswa, Laravel juga membuat tabel bawaan seperti `users`, `cache`, dan `jobs`, serta tabel `migrations` yang mencatat migration mana saja yang sudah dieksekusi. Untuk membatalkan migration terakhir bisa memakai `php artisan migrate:rollback`.

Hasilnya saya periksa di phpMyAdmin dan tabel mahasiswa sudah terbentuk dengan kolom `id`, `nama`, `email`, `jurusan`, `umur`, `created_at`, dan `updated_at`.

C.4 Membuat Model

Model adalah class yang mewakili satu tabel di database. Lewat model, data bisa disimpan dan diambil tanpa menulis perintah SQL secara langsung karena Laravel memakai Eloquent ORM. Model dibuat dengan perintah berikut.

```
php artisan make:model Mahasiswa
```

Perintah itu menghasilkan file `app/Models/Mahasiswa.php`. Nama model ditulis dalam bentuk tunggal dan diawali huruf kapital. Isinya kemudian saya ganti dengan kode berikut.

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Mahasiswa extends Model
8  {
9      protected $table = 'mahasiswa';
10
11     protected $fillable = ['nama', 'nim', 'jurusan'];
12 }
13
```

Properti `$table` diisi `'mahasiswa'` supaya model terhubung ke tabel yang benar, karena secara bawaan Laravel menganggap nama tabel adalah bentuk jamak dari nama model. Properti `$fillable` berisi daftar kolom yang boleh diisi sekaligus lewat mass assignment. Kalau ada kolom yang tidak dicantumkan di `$fillable`, Laravel akan menolak pengisiannya dan menampilkan `MassAssignmentException`.

C.5 Membuat Controller

Controller menjadi penghubung antara view dan model. Controller menerima permintaan pengguna, meminta model menyimpan atau mengambil data, lalu mengirim hasilnya ke view. Saya membuat controller dengan perintah berikut.

```
php artisan make:controller MahasiswaController
```

Perintah itu membuat file `app/Http/Controllers/MahasiswaController.php` dan menampilkan pesan `created successfully`. Isi file kemudian saya ganti dengan kode berikut.

```
1  <?php
2
3  namespace App\Http\Controllers;
4  use App\Models\Mahasiswa;
5  use Illuminate\Http\Request;
6
7  class MahasiswaController extends Controller
8  {
9      public function form(){
10         return view ('form');
11     }
12
13     public function simpan (Request $request){
14         Mahasiswa::create([
15             'nama' => $request->nama,
16             'nim' => $request->nim,
17             'jurusan' => $request->jurusan,
18         ]);
19
20         return "Data mahasiswa berhasil ditambahkan";
21     }
22
23     public function daftar(){
24         $data = Mahasiswa::all();
25         return view('mahasiswa', ['mahasiswa' => $data]);
26     }
27 }
28
```

Di dalam controller ada tiga method. Method `form()` menampilkan halaman input, method `simpan()` menyimpan data dari form ke database lewat `Mahasiswa::create()`, dan method `daftar()` mengambil seluruh data dengan `Mahasiswa::all()` lalu mengirimkannya ke view. Model

Mahasiswa diimpor di bagian atas file supaya bisa dipakai, sedangkan data dari form dibaca melalui objek Request. Berbeda dengan Modul 3 yang datanya masih berupa array, di modul ini data benar-benar disimpan ke dan diambil dari database.

C.6 Routing

Route menghubungkan alamat URL dengan method pada controller. Saya menambahkan tiga route pada file routes/web.php.

```
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\MahasiswaController;
5
6  Route::get('/form', [MahasiswaController::class, 'form']);
7  Route::post('/simpan', [MahasiswaController::class, 'simpan']);
8  Route::get('/daftar-mahasiswa', [MahasiswaController::class, 'daftar']);
9
```

Route /form memakai Route::get karena hanya menampilkan halaman, route /simpan memakai Route::post karena menerima data yang dikirim form, dan route /daftar-mahasiswa memakai Route::get untuk menampilkan data. Controller juga perlu diimpor di bagian atas file dengan use App\Http\Controllers\MahasiswaController; agar bisa dikenali.

C.7 Membuat View Form

View adalah bagian yang dilihat pengguna. File pertama adalah resources/views/form.blade.php untuk menginput data mahasiswa.

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Form Input Mahasiswa</title>
7  </head>
8  <body>
9      <h2>Form Input Mahasiswa</h2>
10
11     <form action="/simpan" method="POST">
12         @csrf
13         <label>Nama</label><br>
14         <input type="text" name='nama'> <br> <br>
15
16         <label>NIM</label><br>
17         <input type="text" name='nim'> <br> <br>
18
19         <label>Jurusan</label><br>
20         <input type="text" name='jurusan'> <br> <br>
21
22         <button type="submit">Simpan</button>
23     </form>
24 </body>
25 </html>
26

```

Form dikirim dengan method POST ke action /simpan. Directive @csrf ditulis di dalam form untuk melindungi dari serangan CSRF. Atribut name pada tiap input, yaitu nama, nim, dan jurusan, dipakai controller untuk membaca data lewat \$request->nama, \$request->nim, dan \$request->jurusan. Nama atribut ini sebaiknya sama dengan nama kolom pada tabel dan isi \$fillable.

C.8 Membuat View Daftar Mahasiswa

File kedua adalah resources/views/mahasiswa.blade.php untuk menampilkan data yang sudah tersimpan.

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <title>Daftar Mahasiswa</title>
8 </head>
9
10 <body>
11     <h2>Daftar Mahasiswa</h2>
12     <table border="1" cellpadding="5" cellspacing="0">
13         <tr>
14             <th>ID</th>
15             <th>Nama</th>
16             <th>Email</th>
17             <th>Umur</th>
18             <th>Jurusan</th>
19         </tr>
20         @foreach($mahasiswa as $mhs)
21             <tr>
22                 <td>{{ $mhs->id }}</td>
23                 <td>{{ $mhs->nama }}</td>
24                 <td>{{ $mhs->email }}</td>
25                 <td>{{ $mhs->umur }}</td>
26                 <td>{{ $mhs->jurusan }}</td>
27             </tr>
28         @endforeach
29     </table>
30 </body>
31
32 </html>
33

```

Directive `@foreach` mengulang baris tabel sebanyak jumlah data yang dikirim dari method `daftar()`. Variabel `$mahasiswa` berisi kumpulan data mahasiswa, dan setiap kolom dipanggil dengan tanda panah seperti `$mhs->nama` karena data dari database berbentuk objek, bukan array seperti pada Modul 3.

C.9 Menjalankan Aplikasi

Server Laravel dijalankan dengan perintah `php artisan serve`. Server berjalan di alamat `http://127.0.0.1:8000` dan terminal harus tetap terbuka selama aplikasi dipakai. Apache dan MySQL pada XAMPP juga harus tetap berjalan.

Setelah server jalan, saya membuka `http://127.0.0.1:8000/form` untuk mengisi form, lalu `http://127.0.0.1:8000/daftar-mahasiswa` untuk melihat daftar. Hasil pengujian dan kendala yang muncul dibahas pada bagian D dan E.

C.10 Membuat Seeder

Setelah tabel dibuat, isinya masih kosong. Untuk mengisi data contoh dalam jumlah banyak, Laravel menyediakan database seeding. Seeder disimpan di folder database/seeder dan dijalankan dengan php artisan db:seed. Migration bisa dibayangkan sebagai membangun rak, sedangkan seeder adalah kegiatan mengisi rak tersebut dengan buku.

Saya membuat seeder dengan perintah berikut.

```
php artisan make:seeder StudentSeeder
```

Perintah itu membuat file database/seeder/StudentSeeder.php. Di folder yang sama juga ada DatabaseSeeder.php, yaitu seeder utama bawaan Laravel yang bisa memanggil seeder lain lewat `$this->call([StudentSeeder::class]);` di dalam method `run()`.

C.11 Membuat Data Dummy dengan Faker

Supaya tidak perlu menulis data satu per satu, saya memakai Faker. Faker adalah library PHP yang sudah terpasang bersama Laravel dan bisa menghasilkan data palsu seperti nama orang, email, dan angka. Faker mendukung locale id_ID sehingga nama yang dihasilkan terdengar lokal. Isi StudentSeeder.php saya ganti dengan kode berikut.

```
1  <?php
2
3  namespace Database\Seeders;
4
5  use Illuminate\Database\Console\Seeds\WithoutModelEvents;
6  use Illuminate\Database\Seeder;
7  use Illuminate\Support\Facades\DB;
8  use Faker\Factory as Faker;
9
10 class StudentSeeder extends Seeder
11 {
12     public function run(): void
13     {
14         $faker = Faker::create('id_ID');
15
16         for ($i = 0; $i < 20; $i++) {
17             DB::table('mahasiswa')->insert([
18                 'nama' => $faker->name,
19                 'email' => $faker->unique()->safeEmail,
20                 'jurusan' => $faker->randomElement(['SIKC', 'Informatika', 'IT']),
21                 'umur' => $faker->numberBetween(18,25),
22                 'created_at' => now(),
23                 'updated_at' => now(),
24             ]);
25         }
26     }
27 }
28
```

`Faker::create('id_ID')` membuat objek Faker dengan format Indonesia. Perulangan `for ($i = 0;`

\$i < 20; \$i++) menghasilkan 20 data. Setiap baris disimpan ke tabel mahasiswa memakai `DB::table('mahasiswa')->insert([...])`. Kolom nama diisi `$faker->name`, email diisi `$faker->unique()->safeEmail` supaya tidak kembar, jurusan dipilih acak dari daftar SIKC, Informatika, dan IT dengan `randomElement`, dan umur diisi angka acak 18 sampai 25 dengan `numberBetween`. Kolom `created_at` dan `updated_at` diisi `now()`.

Sebelum seeder dijalankan, tabel mahasiswa perlu disesuaikan lebih dulu karena seeder mengisi kolom email dan umur, sedangkan migration awal masih memakai kolom nim. Sesuai petunjuk modul, kolom nim diganti menjadi email yang unik dan ditambahkan kolom umur, seperti terlihat pada kode migration di bagian C.3. Setelah itu tabel dibuat ulang dengan php artisan `migrate:refresh`, lalu seeder dijalankan dengan php artisan `db:seed --class=StudentSeeder`. Kedua langkah itu sebenarnya bisa digabung menjadi php artisan `migrate:fresh --seed`.

Setiap kali seeder dijalankan, Laravel menambahkan 20 data baru. Karena datanya acak, nama dan email yang muncul akan berbeda di tiap komputer. Pada project ini seeder sudah dijalankan beberapa kali sehingga tabel mahasiswa berisi 80 baris data.

D. Hasil dan Dokumentasi Program

Berikut tangkapan layar dari aplikasi yang sudah dijalankan dan dari phpMyAdmin.

The screenshot shows the phpMyAdmin interface for the 'mahasiswa' table. The table structure is as follows:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	nama	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	email	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
4	jurusan	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
5	umur	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
6	created_at	timestamp			Yes	NULL			Change Drop More
7	updated_at	timestamp			Yes	NULL			Change Drop More

The 'Indexes' section shows two indexes:

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	80	A	No	
Edit Rename Drop	mahasiswa_email_unique	BTREE	Yes	No	email	80	A	No	

The 'Partitions' section shows 'No partitioning defined!'. The 'Information' section shows the following details:

Space usage		Row statistics	
Data	16.0 KIB	Format	dynamic
Index	16.0 KIB	Collation	utf8mb4_unicode_ci
Overhead	0 B	Next autoindex	81
Console	32.0 KIB	Creation	Oct 05, 2026 at 09:15 AM

Gambar 1. Struktur tabel mahasiswa di phpMyAdmin setelah migration dijalankan.

Form Input Mahasiswa

Nama

NIM

Jurusan

Gambar 2. Halaman form input mahasiswa pada alamat /form.

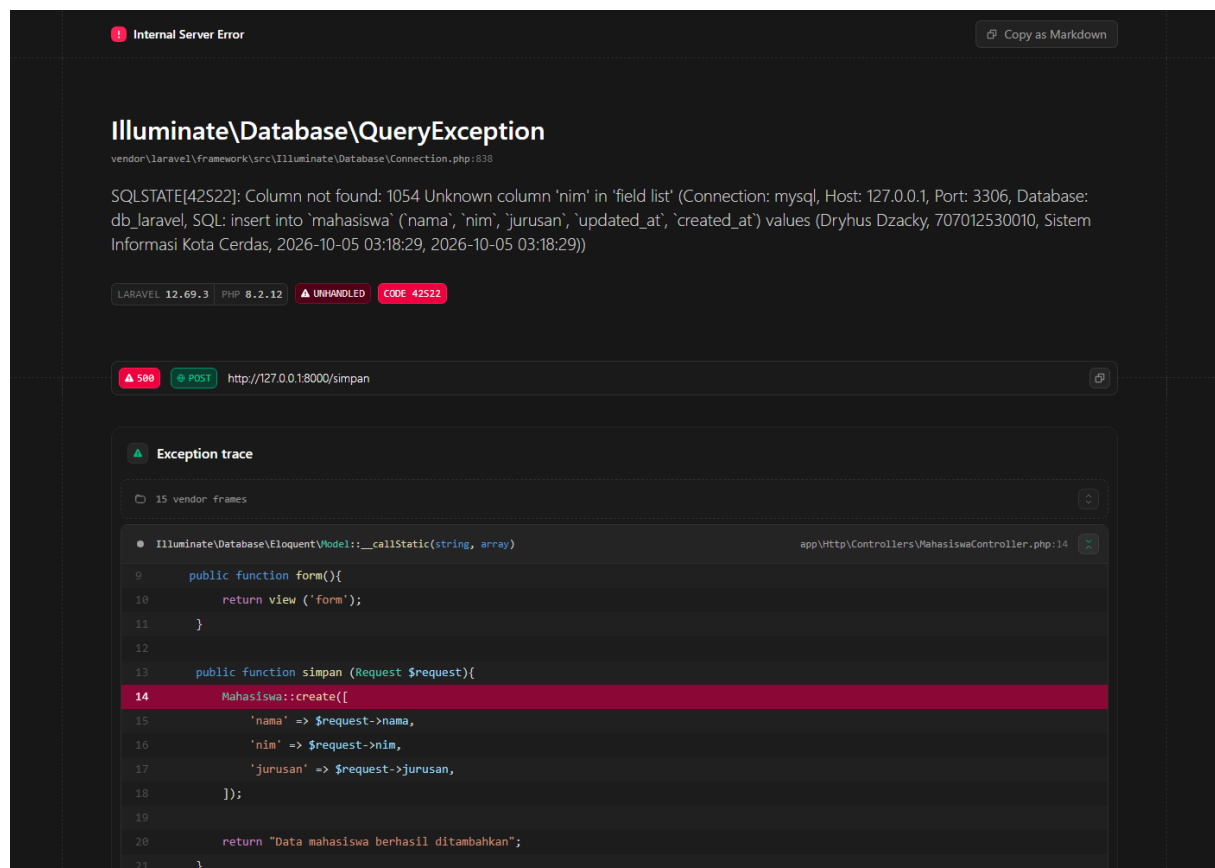
Form Input Mahasiswa

Nama

NIM

Jurusan

Gambar 3. Form yang sudah diisi nama, NIM, dan jurusan sebelum dikirim.



Gambar 4. Galat Unknown column 'nim' yang muncul setelah form dikirim ke /simpan.

Daftar Mahasiswa

ID	Nama	Email	Umur	Jurusan
1	Kani Yuliarti	margana82@example.org	22	SIKC
2	Galar Hendra Habibi M.Ak	bnashiruddin@example.com	24	IT
3	Jaka Kasura Marpaung	winarsih.tami@example.org	22	IT
4	Queen Wahyuni	nova.nugroho@example.net	19	SIKC
5	Prayitna Wibisono	muni29@example.com	18	IT
6	Qori Padmasari S.I.Kom	pbudiyanto@example.net	24	Informatika
7	Kenzie Elon Siregar S.Kom	amelia03@example.org	25	SIKC
8	Ade Yuliarti	asman54@example.org	21	IT
9	Galiono Irsad Pradana S.Gz	gambira71@example.org	23	IT
10	Natalia Hariyah	cinthia.susanti@example.org	21	SIKC
11	Putri Rahayu	usyi18@example.org	22	SIKC
12	Ismail Wacana	bhalimah@example.com	24	IT
13	Jaka Dimaz Wacana	gaiman.pangestu@example.net	24	IT
14	Padma Laksmiwati	hidayat.raharja@example.org	18	SIKC
15	Umayya Putra	handayani.kani@example.org	19	Informatika
16	Bancar Gatot Mustofa	pandu.pudjiastuti@example.net	22	SIKC
17	Halima Palastr	siska.oktaviani@example.org	19	Informatika
18	Endah Gina Wijayanti	dasa.sitorus@example.org	24	IT
19	Mulyono Widodo	raden88@example.org	18	IT
20	Arsipatra Vinsen Budiman S.I.Kom	kuswandari.mursita@example.org	18	Informatika
21	Bancar Narpati S.H.	rosman.namaga@example.net	22	Informatika
22	Gara Nainggolan	sari.rajata@example.org	19	Informatika
23	Harsana Pradipta	wawan58@example.com	23	IT
24	Luluh Budiyanto	uli06@example.com	20	Informatika
25	Padma Nuraini	laras39@example.com	19	IT
26	Cornelia Laksita	shutagalung@example.net	20	SIKC
27	Johan Kamal Tamba	halimah.ratna@example.net	24	Informatika
28	Tira Puput Purwanti M.Ak	riyanti.ida@example.org	21	IT
29	Clara Winarsih	candra98@example.net	22	SIKC
30	Irsad Nashiruddin	vsalahudin@example.net	22	Informatika

Gambar 5. Halaman daftar mahasiswa yang datanya diambil dari database.

Showing rows 0 - 24 (80 total, Query took 0.0004 seconds)

```
SELECT * FROM `mahasiswa`
```

1 > >> Show all Number of rows: 25 Filter rows: Search this table Sort by key: None

	id	nama	email	jurusan	umur	created_at	updated_at
☐	1	Kani Yuliarti	margana82@example.org	SIKC	22	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	2	Galar Hendra Habibi M.Ak	bnashiruddin@example.com	IT	24	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	3	Jaka Kasusra Marpaung	winarsih.tami@example.org	IT	22	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	4	Queen Wahyuni	nova.nugroho@example.net	SIKC	19	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	5	Prayitna Wibisono	muni29@example.com	IT	18	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	6	Qori Padmasari S.I Kom	pbudyanto@example.net	Informatika	24	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	7	Kenzle Elon Siregar S.Kom	amella03@example.org	SIKC	25	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	8	Ade Yuliarti	asman54@example.org	IT	21	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	9	Galliono Irsad Pradana S.Gz	gambira71@example.org	IT	23	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	10	Natalia Hariyah	cinthia.susanti@example.org	SIKC	21	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	11	Putri Rahayu	usyi18@example.org	SIKC	22	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	12	Ismail Wacana	bhalimah@example.com	IT	24	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	13	Jaka Dimaz Wacana	gaiman.pangestu@example.net	IT	24	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	14	Padma Laksmiwati	hidayat.raharja@example.org	SIKC	18	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	15	Umayya Putra	handayani.kani@example.org	Informatika	19	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	16	Bancar Gatot Mustofa	pandu.pudjastuti@example.net	SIKC	22	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	17	Halima Palastri	siska.oktaviani@example.org	Informatika	19	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	18	Endah Gina Wijayanti	dasa.sitorus@example.org	IT	24	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	19	Mulyono Widodo	raden88@example.org	IT	18	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	20	Arsipatra Vinsen Budiman S.I Kom	kuswandari.mursita@example.org	Informatika	18	2026-10-05 02:15:46	2026-10-05 02:15:46
☐	21	Bancar Narpati S.H.	rosman.namaga@example.net	Informatika	22	2026-10-05 02:16:34	2026-10-05 02:16:34
☐	22	Gara Nainggolan	sari.rajata@example.org	Informatika	19	2026-10-05 02:16:34	2026-10-05 02:16:34
☐	23	Harsana Pradipta	wawan58@example.com	IT	23	2026-10-05 02:16:34	2026-10-05 02:16:34
☐	24	Luluh Budyanto	uli06@example.com	Informatika	20	2026-10-05 02:16:34	2026-10-05 02:16:34
☐	25	Padma Nuraini	laras39@example.com	IT	19	2026-10-05 02:16:34	2026-10-05 02:16:34

Gambar 6. Data hasil seeder pada tabel mahasiswa di phpMyAdmin.

E. Analisis dan Pembahasan

E.1 Peran Migration dan Seeder

Migration dan seeder punya tugas yang berbeda meskipun keduanya berhubungan dengan database. Migration mengurus struktur tabel, misalnya menentukan nama tabel dan tipe tiap kolom. Seeder mengurus isi tabel, yaitu memasukkan data awal atau data contoh. Karena itu urutannya selalu migration dulu, baru seeder. Pada praktikum ini migration membuat tabel mahasiswa, sedangkan StudentSeeder mengisi tabel tersebut dengan 20 data dummy.

File migration juga menyimpan riwayat perubahan struktur. Kalau suatu saat kolom diganti, perubahan itu ditulis di file migration baru dan bisa dijalankan ulang di komputer lain dengan php artisan migrate. Seeder tidak menyimpan riwayat seperti itu, karena fungsinya hanya mengisi data dan bisa dijalankan berkali-kali tanpa mengubah struktur.

E.2 Perbedaan Aplikasi Sebelum dan Sesudah Memakai Database

Pada Modul 3, data laporan masih ditulis sebagai array di dalam controller. Data seperti itu hanya hidup selama halaman dibuka dan hilang ketika halaman ditutup atau server dimatikan. Di modul ini data disimpan ke tabel MySQL melalui model Eloquent, sehingga data tetap ada meskipun server dijalankan ulang.

Perubahan lain terlihat pada cara kerja controller dan view. Method simpan() sekarang memanggil Mahasiswa::create() untuk menulis baris baru ke tabel, sedangkan method daftar()

memakai Mahasiswa::all() untuk membaca seluruh data. Di sisi view, data dari database berbentuk objek sehingga kolomnya dipanggil dengan \$mhs->nama, bukan \$mhs['nama'] seperti pada array. Laravel juga menerjemahkan perintah Eloquent menjadi SQL secara otomatis, jadi tidak ada perintah INSERT atau SELECT yang ditulis manual.

E.3 Kendala dan Penyelesaian

1. Galat Unknown column 'nim' saat form dikirim. Setelah kolom nim pada migration diganti menjadi email dan ditambahkan kolom umur, saya belum menyesuaikan bagian lain yang masih memakai nim. Akibatnya ketika form dikirim ke /simpan, Laravel menjalankan INSERT dengan kolom nim yang sudah tidak ada di tabel dan muncul pesan SQLSTATE[42S22]: Column not found: 1054 Unknown column 'nim'. Ini terlihat pada Gambar 4. Penyelesaiannya adalah menyesuaikan isi \$fillable pada model Mahasiswa, method simpan() pada MahasiswaController, dan input pada form.blade.php agar memakai kolom email dan umur. Modul memang mengingatkan bahwa setelah kolom nim diganti, bagian-bagian tersebut harus ikut disesuaikan kalau form dan halaman daftar ingin tetap dipakai. Pada kondisi terakhir project, penyesuaian itu belum diterapkan, sehingga halaman daftar mahasiswa tetap bisa menampilkan data hasil seeder sementara alur simpan dari form masih gagal.

2. Tabel mahasiswa sempat tidak cocok dengan seeder. Seeder mengisi kolom email dan umur, sedangkan migration awal hanya punya kolom nim. Kalau seeder dijalankan tanpa mengubah migration, akan muncul galat kolom tidak ditemukan. Masalah ini diselesaikan dengan mengganti kolom nim menjadi email dan menambahkan kolom umur pada file migration, lalu menjalankan php artisan migrate:refresh supaya tabel dibentuk ulang dengan struktur baru. Perlu diingat bahwa perintah ini menghapus seluruh data lama, jadi hanya cocok dipakai saat pengembangan.

F. Kesimpulan

1. Koneksi database Laravel berhasil diatur melalui file .env dengan DB_CONNECTION mysql dan DB_DATABASE db_laravel, sehingga aplikasi bisa membaca dan menulis ke database MySQL.
2. Migration berhasil membuat tabel mahasiswa beserta kolomnya, dan model Eloquent Mahasiswa berhasil menghubungkan tabel itu ke aplikasi lewat properti \$table dan \$fillable.
3. Alur form, route, controller, model, dan database sudah tersambung. Halaman daftar mahasiswa menampilkan data dari database, sedangkan seeder dengan Faker berhasil mengisi tabel dengan data dummy secara otomatis.
4. Praktikum ini menunjukkan pentingnya menjaga kesesuaian antara struktur tabel dan kode

yang memakainya. Ketika kolom nim diganti menjadi email dan umur, bagian model, controller, dan form harus ikut diperbarui supaya tidak muncul galat kolom tidak ditemukan.

G. Link Repository Github

Kode program lengkap praktikum ini dapat diakses pada repositori GitHub berikut:

<https://github.com/Dryhus21/PABW.git>